

NAME: ALIGE OYE

Lab Environ : kali/DVWA

Date:15/08/2026

This is a practical project report covering SQLi(low, medium, high and impossible) in dvwa and Bash automation of hashing a file.

2 Low Security Level — SQL Injection Exploitation

SQL injection occurs when an application incorporates untrusted user input directly into an SQL statement, allowing the input to modify the intended SQL logic. At the Low security level of DVWA, the supplied `id` value is incorporated into the SQL query without adequate validation or parameterization. Consequently, SQL syntax supplied through the input can be interpreted by the database as part of the query.

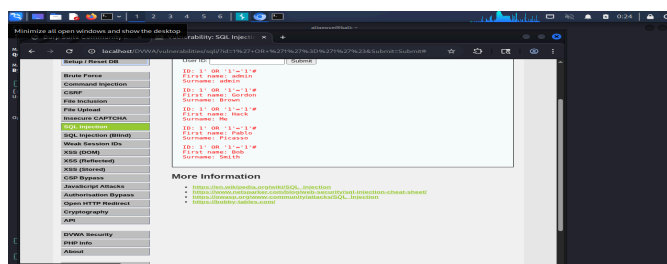
The exploitation was performed in the following sequence.

i. Injection confirmation

The payload:

`1' OR '1'='1'#`

was submitted. The single quote terminates the original string, while `OR '1'='1'` introduces a condition that is always true. The `#` begins a SQL comment, causing the remainder of the original query to be ignored. The application consequently returned multiple records, confirming that the parameter was vulnerable to SQL injection.



ii. Column-count determination

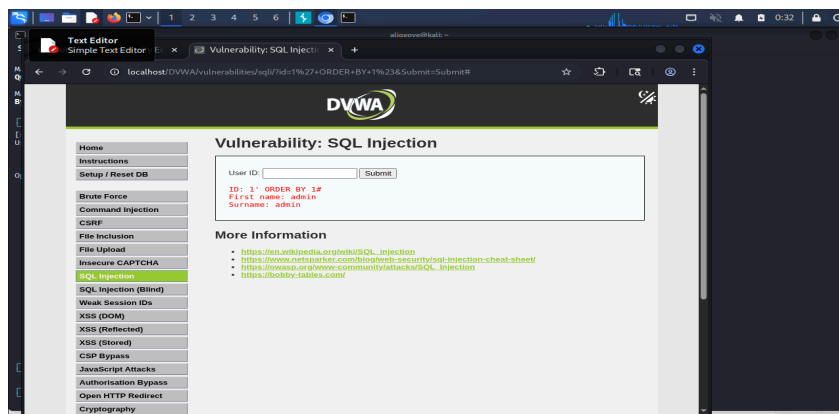
The following payloads were tested sequentially:

1' ORDER BY 1#

1' ORDER BY 2#

1' ORDER BY 3#

The responses showed that the underlying query returns two columns. This information is necessary because a UNION SELECT must return the same number of columns as the original query.



iii. UNION column mapping

The payload:

1' UNION SELECT 'A','B'#

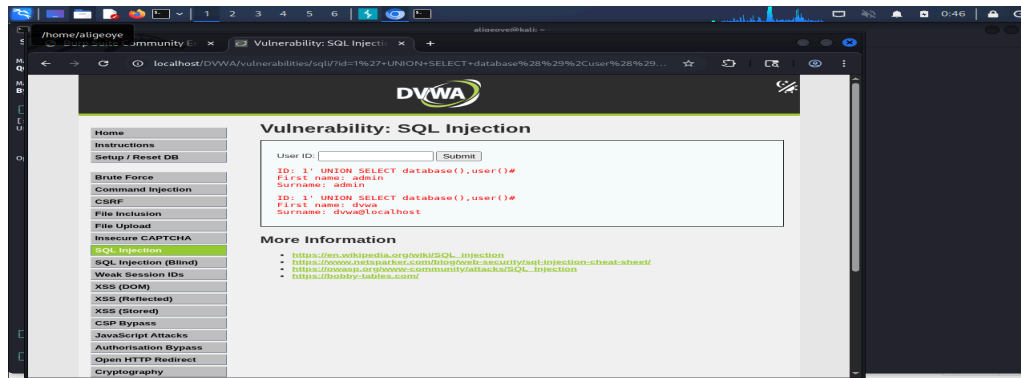
was used to identify the output positions of the two columns. The returned values demonstrated which columns were reflected in the application's response.

iv. Database enumeration

The following payload was then used:

1' UNION SELECT database(),user()#

This caused the database to return the current database name and the database account being used to execute the query. This demonstrates that the vulnerability can be used for information gathering beyond the original user-ID lookup.



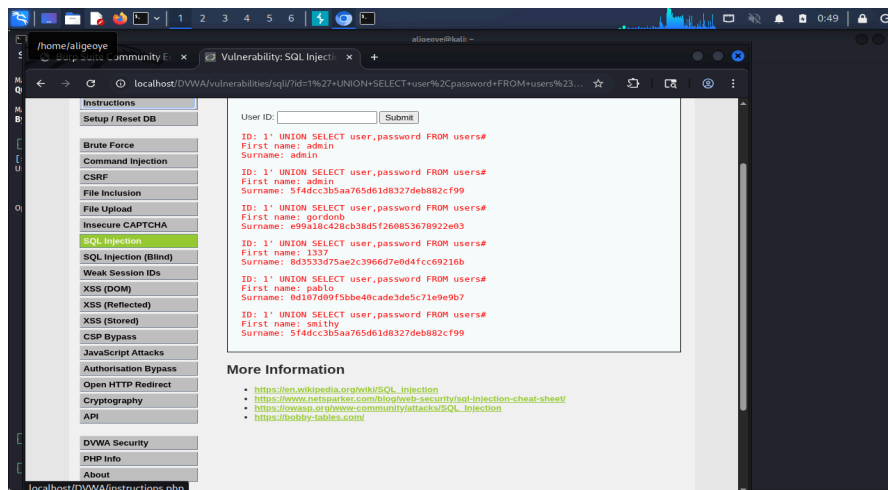
v. Credential extraction

Finally, the following payload was used against the DVWA training database:

```
1' UNION SELECT user,password FROM users#
```

This returned the usernames and password hashes stored in the `users` table.

The attack succeeds because the application allows attacker-controlled input to become part of the SQL syntax. The attacker can therefore terminate the original string, inject additional SQL expressions, and comment out the remaining portion of the query. The key security failure is the failure to separate SQL instructions from user-supplied data.

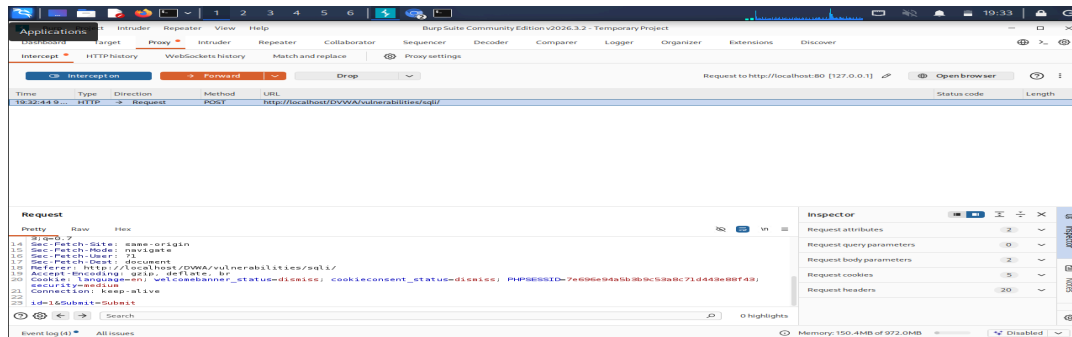


2(b)(b) Medium Security Level — POST Parameter and Numeric SQL Injection

1. At the Medium security level, the SQL Injection input is presented as a dropdown and submitted to the application through a POST request. Burp Suite is therefore useful because it allows the HTTP request generated by the legitimate dropdown selection to be intercepted and modified before it reaches the server. For example, the request contains a parameter similar to:

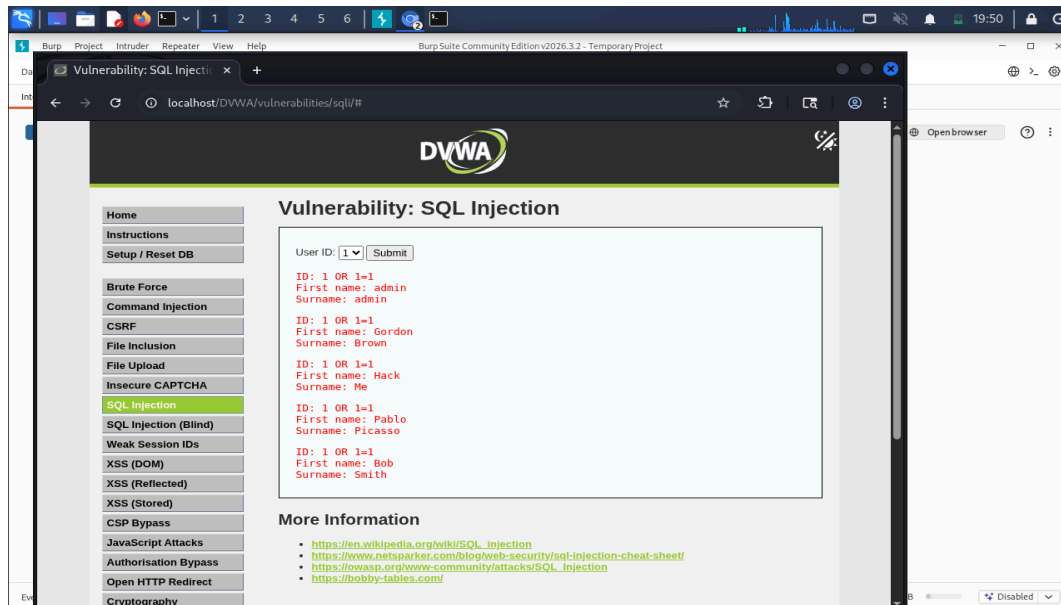
id=1&Submit=Submit

The **id** value can be modified in Burp Suite even though the browser's dropdown only provides legitimate numeric choices.



The important weakness is that the application escapes quotation marks, but the SQL parameter is being used in a numeric context. Quote escaping does not prevent an expression such as:

1 OR 1=1



because this payload contains no quotation mark. Conceptually, the resulting query becomes:

```
SELECT first_name, last_name
```

```
FROM users
```

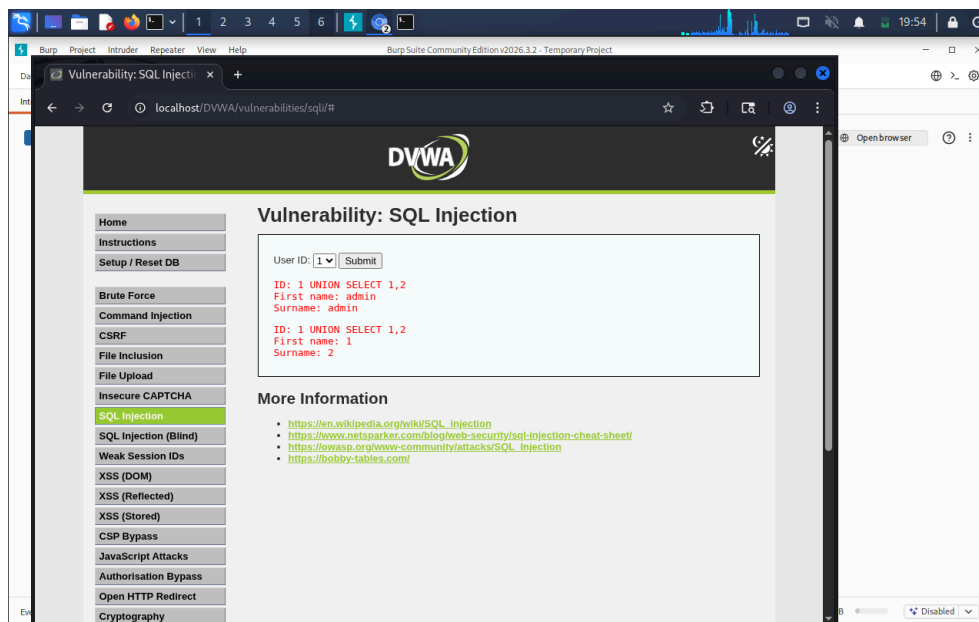
```
WHERE user_id = 1 OR 1=1;
```

Since `1=1` is always true, the condition can return multiple records. This demonstrates that escaping quotes is not a complete defense against SQL injection.

2. A UNION test can also be performed by modifying the POST parameter in Burp:

```
1 UNION SELECT 1,2
```

The two values correspond to the two columns returned by the underlying DVWA query. The fact that this can be performed without quotation marks further demonstrates the limitation of quote escaping.

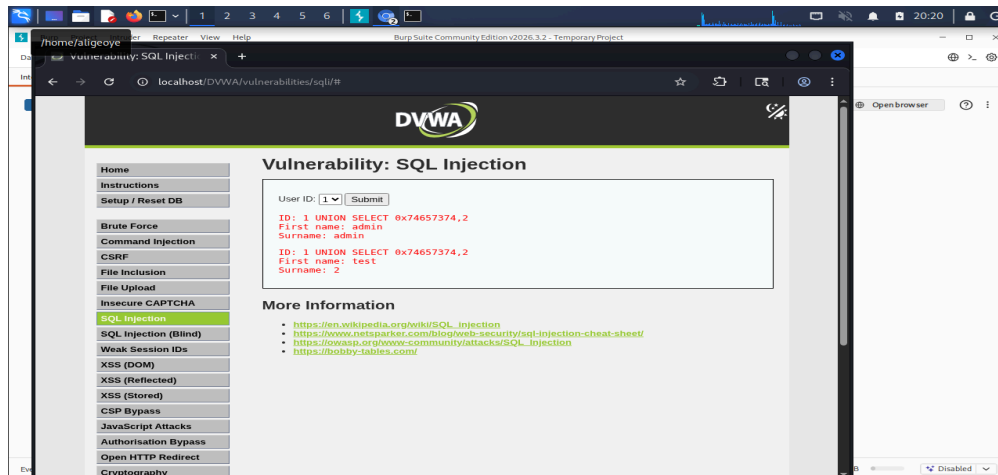


3. Hexadecimal encoding. SQL hexadecimal literals allow a string value to be represented without surrounding quotation marks. For example:

```
0x74657374
```

represents the string `test`. Therefore, a UNION request can contain:

```
1 UNION SELECT 0x74657374,2
```



No single quote is required. This demonstrates that simply escaping quotes does not reliably separate SQL code from user-controlled data.

The correct security lesson is that quote escaping should not be relied upon as the primary SQL injection defense. User input should instead be supplied through parameterized/prepared SQL statements so that the database treats the input as data rather than executable SQL syntax.

2(c) High Security Level — Session Input and LIMIT 1

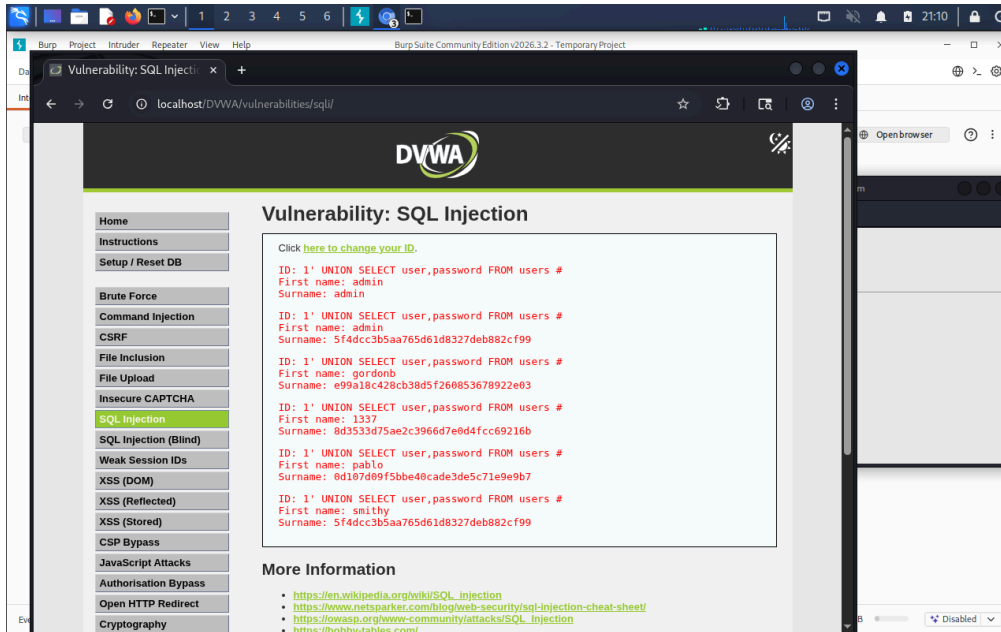
At the High security level, the user input is stored in the server-side session before being used by the SQL Injection page. The application also adds a **LIMIT 1** clause to the SQL query in an attempt to restrict the result to one record. However, the application remains vulnerable because the session value is still concatenated directly into the SQL statement.

The following payload successfully demonstrated the vulnerability:

```
1' UNION SELECT user,password FROM users #
```

The payload first closes the original quoted value with `'`. The **UNION SELECT** then adds a query that retrieves the `user` and `password` columns from the `users` table. Finally, `#` begins a MySQL comment. Consequently, the remainder of the application's SQL statement, including the trailing **LIMIT 1**, is ignored.

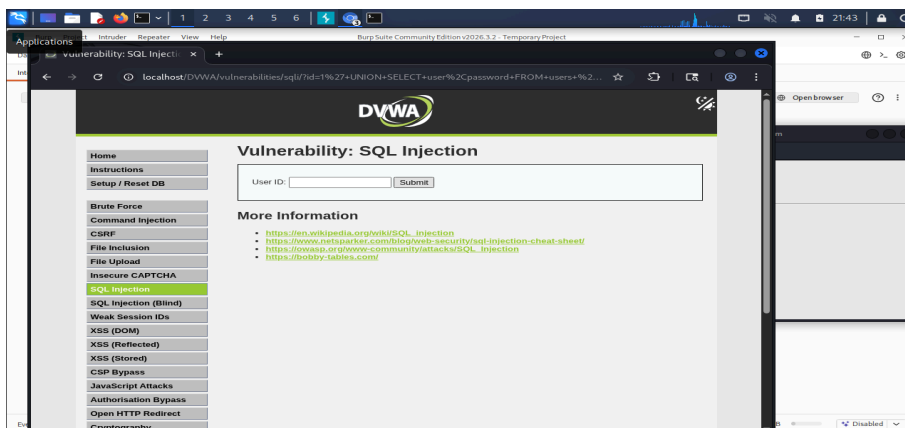
The successful result displayed multiple usernames and their corresponding password hashes, proving that adding **LIMIT 1** and storing the input in a session **does not constitute protection against SQL injection**.



2(d) Impossible Security Level

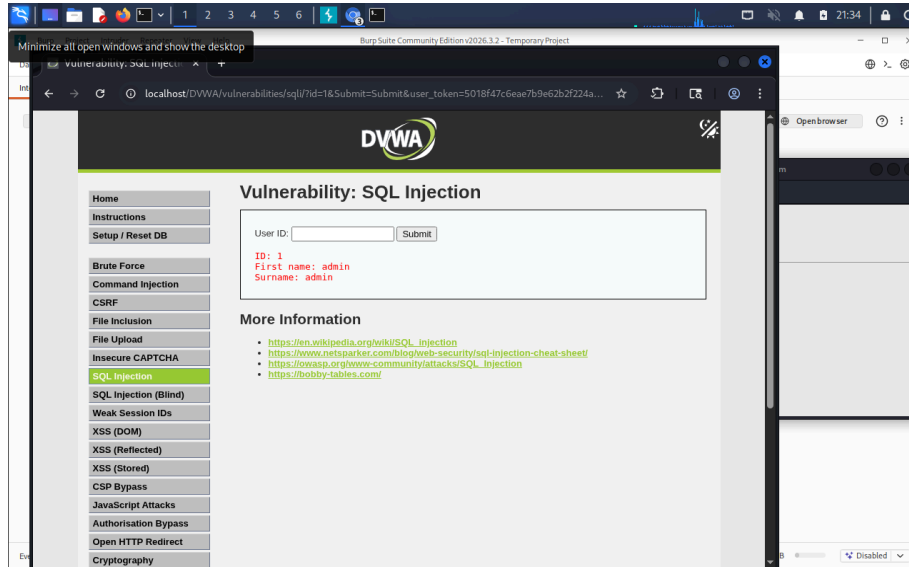
At the Impossible security level, DVWA uses a prepared statement to execute the SQL query. The SQL structure is defined separately from the user-supplied input.

When the SQL injection payload: `1' UNION SELECT user,password FROM users #`



was submitted, it did not execute as SQL code. Instead, the entire input was treated as a data value, so the **UNION** and comment characters could not change the structure of the query. Therefore, the attack fails because prepared statements separate SQL code from user data.

But below is a screenshot of normal 1 request working.



Recommendation to the developer: Use prepared/parameterized statements for all queries involving user-controlled input. Do not rely on quote escaping, hiding queries, sessions, or **LIMIT 1** as the primary SQL injection defense.

2(e) Hash identification and cracking: The recovered password hashes were 32-character hexadecimal values, consistent with **MD5**. John the Ripper identified them as **Raw-MD5**. I created a hash file containing the four recovered hashes and used the RockYou wordlist to perform a dictionary attack:

```
john --format=Raw-MD5 --wordlist=/usr/share/wordlists/rockyou.txt  
hashes.txt
```

The attack successfully cracked all four hashes, recovering the passwords **password**, **abc123**, **letmein**, and **charley**. The results were retrieved using:

```
john --show --format=Raw-MD5 hashes.txt
```



```

No VM guests are running outdated hypervisor (qemu) binaries on this host.

(aligeoye@kali)-[~]
$ john --version
Created directory: /home/aligeoye/.john
Unknown option: "--version"

(aligeoye@kali)-[~]
$ john --format=RAW-MD5 --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt
Using default input encoding: UTF-8
Loaded 4 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
password      (?)
abc123        (?)
letmein       (?)
charley       (?)
4g 0:00:00:00 DONE (2026-08-09 23:17) 50.00g/s 38400p/s 38400c/s 57600C/s my3kids..dangerous
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.

(aligeoye@kali)-[~]
$ john --show --format=Raw-MD5 hashes.txt
?:password
?:abc123
?:charley
?:letmein

4 password hashes cracked, 0 left

```

John stores its working/session data under the user's `.john` directory, shown in this lab as `/home/aligeoye/.john`. The `--show` command can be used later to retrieve the recovered credentials associated with the hashes.

Two specific remediation recommendations

1. **Use prepared statements/parameterized queries** for every database query instead of constructing SQL statements by concatenating user input.
2. **Never store passwords as unsalted MD5 hashes.** Use a modern password-hashing function such as **Argon2id**, **bcrypt**, or **scrypt**, with a unique salt per password.

5(a)i Bash Automation

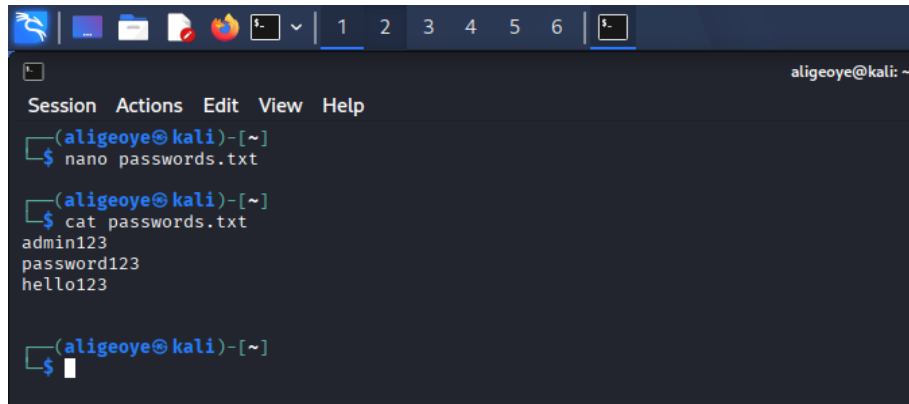
The objective was to automate the repetitive task of hashing a file containing plaintext passwords. A Bash script was created to read each password from an input file, generate its SHA-256 hash, and save the resulting hashes to `hashes.txt`.

I created a file name `passwords.txt` containing the below passwords;

admin123

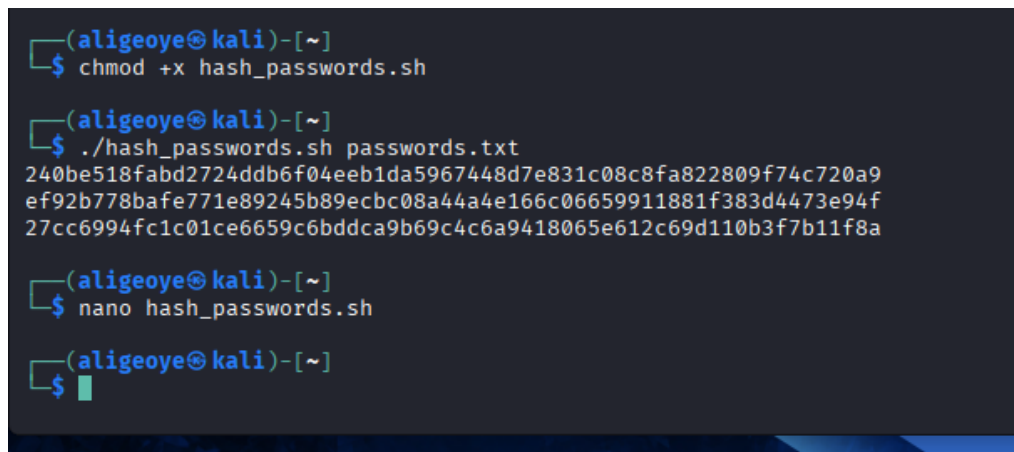
password123

Hello123

A terminal window on a Kali Linux system. The window title is 'aligeoye@kali: ~'. The menu bar shows 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal shows the following commands and output:

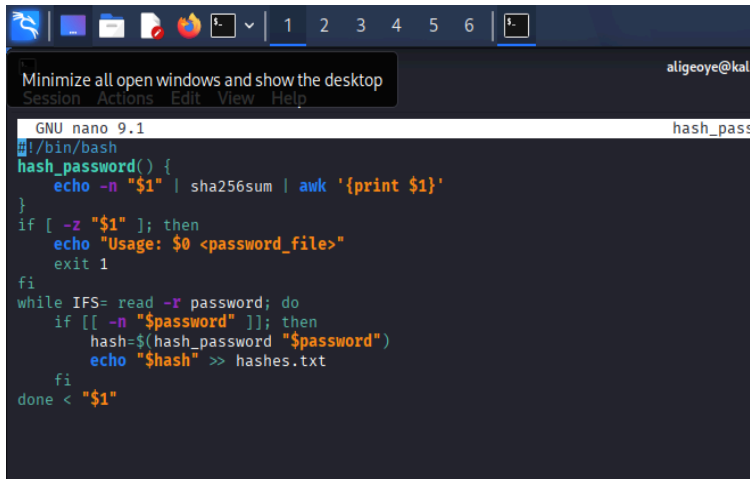
```
(aligeoye@kali)-[~]  
$ nano passwords.txt  
  
(aligeoye@kali)-[~]  
$ cat passwords.txt  
admin123  
password123  
hello123  
  
(aligeoye@kali)-[~]  
$
```

It was then output to passwords.sh

A terminal window on a Kali Linux system. The window title is 'aligeoye@kali: ~'. The menu bar shows 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal shows the following commands and output:

```
(aligeoye@kali)-[~]  
$ chmod +x hash_passwords.sh  
  
(aligeoye@kali)-[~]  
$ ./hash_passwords.sh passwords.txt  
240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9  
ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f  
27cc6994fc1c01ce6659c6bddca9b69c4c6a9418065e612c69d110b3f7b11f8a  
  
(aligeoye@kali)-[~]  
$ nano hash_passwords.sh  
  
(aligeoye@kali)-[~]  
$
```

ii) Script used



```
GNU nano 9.1 hash_pass
#!/bin/bash
hash_password() {
    echo -n "$1" | sha256sum | awk '{print $1}'
}
if [ -z "$1" ]; then
    echo "Usage: $0 <password_file>"
    exit 1
fi
while IFS= read -r password; do
    if [[ -n "$password" ]]; then
        hash=$(hash_password "$password")
        echo "$hash" >> hashes.txt
    fi
done < "$1"
```

The script was made executable using: `chmod +x hash_passwords.sh`

It was then executed with: `./hash_passwords.sh passwords.txt`

The generated hash file was viewed using: `cat hashes.txt`

Line-by-line explanation

1. Shebang: `#!/bin/bash`

This is the **shebang**. It tells the operating system to execute the script using the Bash shell.

2. Function: `hash_password() {`

This defines a function named `hash_password`. Creating a function avoids repeating the hashing commands for every password.

3. Safe handling of a missing argument: `if [ -z "$1" ]; then`

This checks whether the first argument is empty. The first argument should be the password file.

4. Loop: `while IFS= read -r password; do`

This creates a loop that reads the input file **one line at a time**. Each line is stored in the `password` variable.

5. Conditional: if [[-n "\$password"]]; then

This checks that the current line is not empty. Empty lines are therefore not hashed.

6. Command substitution: hash=\$(hash_password "\$password")

The `$(...)` syntax is **command substitution**. It runs the `hash_password` function and stores its output in the `hash` variable.

Saving the hashes: `echo "$hash" >> hashes.txt`

This writes each generated hash to `hashes.txt`. The `>>` operator appends each hash as a new line.

Conclusion

The Bash script successfully automated the repetitive hashing process. It demonstrates a **shebang**, **function**, **loop**, **conditional statements**, **command substitution**, and **safe handling of a missing argument**. The resulting `hashes.txt` file contains one SHA-256 hash for each non-empty password in the input file.

5(b) Test the 32-character hash

Step 1 — I Create the MD5 hash file

`nano md5.txt`

Step 2 — I Crack it with john the ripper

`john --format=Raw-MD5 --pot=/tmp/md5-demo.pot --wordlist=/usr/share/wordlists/rockyou.txt md5.txt`

```
1 password hash cracked, 0 left
(aligeoye@kali)-[~]
$ john --format=Raw-MD5 --pot=/tmp/md5-demo.pot --wordlist=/usr/share/wordlists/rockyou.txt md5.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
password (??)
1g 0:00:00:01 DONE (2026-08-13 21:48) 0.7575g/s 290.9p/s 290.9c/s 290.9C/s 123456..michael1
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
(aligeoye@kali)-[~]
$ john --show --format=Raw-MD5 --pot=/tmp/md5-demo.pot md5.txt
?:password
.
1 password hash cracked, 0 left
(aligeoye@kali)-[~]
$
```

HASHCAT WAS UNSUCCESSFUL: REASON; Your Kali repositories currently don't have `pocl-openc1-icd` available. I checked the current Kali package information, and PoCL was removed from `kali-rolling` in March 2026. There is also a current Kali bug report describing Hashcat 7.1.2 failing to find an OpenCL device on Kali 2026.2.

```
(aligeoye@kali)-[~]
$ sudo apt install pocl-openc1-icd
Package pocl-openc1-icd is not available, but is referred to by another package.
This may mean that the package is missing, has been obsoleted, or
is only available from another source

Error: Package 'pocl-openc1-icd' has no installation candidate

(aligeoye@kali)-[~]
```

\$6\$ hash: Step 1

\$6\$ means **SHA-512 crypt**, not ordinary SHA-512.

I create a fresh \$6\$ test hash for the lab.

```
openssl passwd -6 -salt demo password
```

```
nano sha512crypt.txt
```

```
john --format=sha512crypt --wordlist=/usr/share/wordlists/rockyou.txt sha512crpt.txt
```

```
(aligeoye@kali)-[~]
$ openssl passwd -6 -salt demo password
$6$demo$orIFPHZCXTAFh/ygfpjY0Jq2KE3WE570Zhw1RBpRnm0b0w.QG.7RvB.n.qfMQIjutyNLXKr0JpDzHA7QBWL6.

(aligeoye@kali)-[~]
$ nano sha512crypt.txt

(aligeoye@kali)-[~]
$ nano sha512crpt.txt

(aligeoye@kali)-[~]
$ cat sha512crypt.txt
cat: sha512crypt.txt: No such file or directory

(aligeoye@kali)-[~]
$ nano sha512crpt.txt

(aligeoye@kali)-[~]
$ john --format=sha512crypt --wordlist=/usr/share/wordlists/rockyou.txt sha512crpt.txt
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2 4x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
password (?)
1g 0:00:00:00 DONE (2026-08-13 22:51) 3.703g/s 948.1p/s 948.1c/s 948.1C/s 123456..freedom
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(aligeoye@kali)-[~]
```

bcrypt (\$2b\$).

Step 1 — I Generate a bcrypt hash

```
openssl passwd -bcrypt password
```

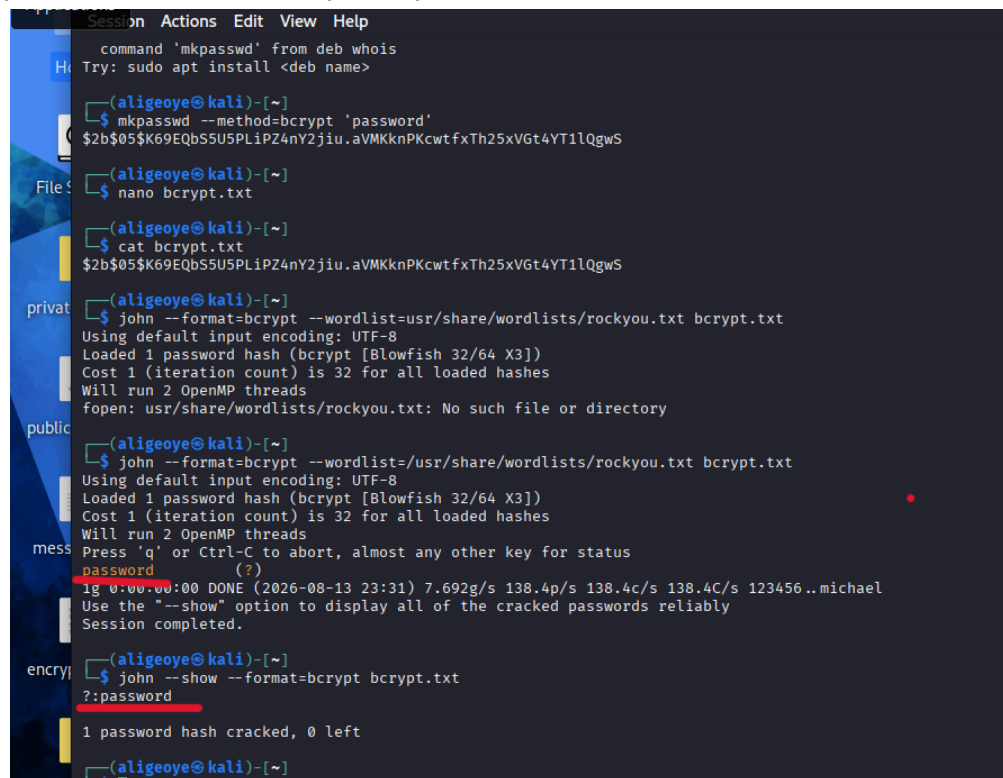
Step 2 — I Saved it

```
nano bcrypt.txt
```

Step 3 — John the Ripper

```
john --format=bcrypt --wordlist=/usr/share/wordlists/rockyou.txt bcrypt.txt
```

```
john --show --format=bcrypt bcrypt.txt
```



```
Session Actions Edit View Help
command 'mkpasswd' from deb whois
Try: sudo apt install <deb name>

(aligeoye@kali)-[~]
$ mkpasswd --method=bcrypt 'password'
$2b$05$K69EQbSS5USPLiPZ4nY2jiu.aVMKknPKcwtfxTh25xVGt4YT1lQgWS

(aligeoye@kali)-[~]
$ nano bcrypt.txt

(aligeoye@kali)-[~]
$ cat bcrypt.txt
$2b$05$K69EQbSS5USPLiPZ4nY2jiu.aVMKknPKcwtfxTh25xVGt4YT1lQgWS

(aligeoye@kali)-[~]
$ john --format=bcrypt --wordlist=/usr/share/wordlists/rockyou.txt bcrypt.txt
Using default input encoding: UTF-8
Loaded 1 password hash (bcrypt [Blowfish 32/64 X3])
Cost 1 (iteration count) is 32 for all loaded hashes
Will run 2 OpenMP threads
fopen: /usr/share/wordlists/rockyou.txt: No such file or directory

(aligeoye@kali)-[~]
$ john --format=bcrypt --wordlist=/usr/share/wordlists/rockyou.txt bcrypt.txt
Using default input encoding: UTF-8
Loaded 1 password hash (bcrypt [Blowfish 32/64 X3])
Cost 1 (iteration count) is 32 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
password (?)
ig 0:00:00:00 DONE (2026-08-13 23:31) 7.692g/s 138.4p/s 138.4c/s 138.4C/s 123456..michael
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(aligeoye@kali)-[~]
$ john --show --format=bcrypt bcrypt.txt
?:password

1 password hash cracked, 0 left

(aligeoye@kali)-[~]
```

Where the cracked passwords are stored

John the Ripper normally stores recovered passwords in its **pot file**, commonly:

```
~/.john/john.pot
```

The recovered passwords can conveniently be displayed with:

```
john --show hashfile.txt
```

Hashcat normally maintains a **potfile** containing recovered hashes and their corresponding passwords. The location can depend on the installation/configuration; it can be checked with:

```
hashcat --show
```

“Nothing left to crack” explanation

During testing, John may display:

```
No password hashes left to crack
```

It usually means John has already recovered that hash previously and the result is already stored in its pot file. John therefore does not need to perform the same cracking operation again.

For a clean demonstration, specify a **new temporary pot file** using `--pot`:

```
john --format=Raw-MD5 --pot=/tmp/md5-demo.pot \ --wordlist=/usr/share/wordlists/rockyou.txt  
md5.txt
```

5(c) Explanation and Recommendations

Why password length beats complexity in offline cracking

Offline password cracking occurs when an attacker obtains password hashes and can test password guesses without interacting with the original login system. The attacker repeatedly hashes candidate passwords using the appropriate algorithm and compares the resulting hash with the stolen hash.

Password length is generally more important than simply adding complexity because every additional character increases the number of possible combinations. For example, a short

password containing uppercase letters, numbers and symbols can still have a relatively small search space. A long passphrase has a much larger search space, making exhaustive guessing significantly more difficult.

Attackers also use dictionaries and password lists containing common passwords and predictable substitutions such as Password123!. Therefore, simply adding a capital letter, number or symbol to a short predictable password does not provide as much protection as using a long, unique and unpredictable passphrase.

Three prioritized recommendations

1. Enforce long, unique passwords/passphrases — Highest priority

The client should require sufficiently long passwords and encourage passphrases rather than relying mainly on complexity rules. Passwords should also be unique for each account so that a compromised password cannot be reused against another service.

2. Use strong, password-specific hashing with salts — High priority

Passwords should never be stored as plaintext or using fast hashes such as MD5 or SHA-1. The client should use a modern password hashing algorithm such as Argon2id, bcrypt, or an appropriately configured scrypt/PBKDF2 implementation, with a unique salt for every password. These algorithms deliberately make password guessing more expensive.

3. Implement multi-factor authentication (MFA) — High priority

MFA should be enabled, particularly for administrator, financial and other high-value accounts. Even if an attacker successfully cracks a password offline, an additional authentication factor can prevent the attacker from logging into the account.

Conclusion

The key defence against offline password cracking is to increase the attacker's cost per guess and increase the size of the password search space. Long, unique passwords combined with slow password hashing and MFA provide substantially stronger protection than relying on short passwords with complicated character requirements alone.

